Technical Details of V3DB

1 OVERVIEW

This note provides additional technical details for the main paper and uses the same notation throughout. Its purpose is not to introduce new claims, but to record the complete technical constructions and the corresponding gate-complexity expansions in one place.

The remainder of the note is organized as follows:

- Section 2 spells out the fixed-shape five-step IVF-PQ semantics in fuller detail.
- Section 3 expands the snapshot-binding circuit and derives the commitment-side overhead G_{commit}.
- Section 4 collects the reusable gadgets used by the later constructions and summarizes their gate costs.
- Section 5 gives the circuit-only baseline design and derives both G_{baseline} and $G^{\text{total}}_{\text{baseline}}$.
- Section 6 gives the multiset-based design and derives both G_{multiset} and $G^{\text{total}}_{\text{multiset}}$.

2 FIVE-STEP IVF-PQ SEMANTICS

The main paper already summarizes the fixed-shape five-step IVF-PQ semantics. This section restates it in fuller detail for two reasons. First, the main paper is space-constrained, so we expand the semantic objects and intermediate sequences more explicitly here. Second, the later sections of this note all build on the same five-step decomposition, so restating it here improves the self-contained readability of the technical presentation. As in the main paper, we use the shorthand $[Z] := \{0, 1, \ldots, Z-1\}$ to denote integer index ranges.

Public parameters and committed snapshot. The public parameters are the raw database size N_0, the padded capacity

$$N := n_{\text{list}} \cdot n,$$

the vector dimension D, the number of inverted lists n_{list}, the number of probed lists n_{probe}, the top-k size k, the per-list capacity n, and the PQ parameters (M, K) with $D = Md$.

The committed snapshot contains:

- the centroid table
$$\boldsymbol{\mu} = (\mu_0, \ldots, \mu_{n_{\text{list}}-1}), \qquad \mu_i \in \mathbb{F}^D;$$
- the PQ codebooks
$$\{C_{m,k}\}_{m\in[M],\, k\in[K]}, \qquad C_{m,k} \in \mathbb{F}^d;$$
- the fixed-shape inverted lists, where each slot stores a record
$$(f_{i,j}, \text{item}_{i,j}, v^{\text{PQ}}_{i,j}), \qquad i \in [n_{\text{list}}],\ j \in [n].$$
Here $f_{i,j} \in \{0, 1\}$ is the validity flag, $\text{item}_{i,j}$ is the application-defined payload, and
$$v^{\text{PQ}}_{i,j} = (v^{(0)}_{i,j}, \ldots, v^{(M-1)}_{i,j}) \in [K]^M$$
is the PQ code. If $f_{i,j} = 0$, the slot is padding and the remaining fields are arbitrary.

We use squared ℓ_2 distance over $\mathbb{F}$ throughout:

$$\text{dist}(x, y) := \sum_{t=0}^{D-1} (x_t - y_t)^2.$$

Step 1. Centroid distances. For a query $q \in \mathbb{F}^D$, the semantics first computes

$$d_i := \text{dist}(q, \mu_i), \qquad i \in [n_{\text{list}}],$$

and forms the base index-distance sequence

$$S^{\text{orig}}_{\text{idx,dist}} := \big((0, d_0), (1, d_1), \ldots, (n_{\text{list}} - 1, d_{n_{\text{list}}-1})\big).$$

This sequence is the arithmetic base object for probe selection.

Step 2. Probe selection. The semantics reorders $S^{\text{orig}}_{\text{idx,dist}}$ by distance and obtains

$$S^{\text{sorted}}_{\text{idx,dist}} := \big((i_0, d_{i_0}), (i_1, d_{i_1}), \ldots, (i_{n_{\text{list}}-1}, d_{i_{n_{\text{list}}-1}})\big),$$

where $(i_0, \ldots, i_{n_{\text{list}}-1})$ is a permutation of $[n_{\text{list}}]$. Only the first n_{probe} entries are required to satisfy the prefix-order and boundary conditions

$$d_{i_0} \le d_{i_1} \le \cdots \le d_{i_{n_{\text{probe}}-1}} \quad \text{and}$$
$$d_{i_{n_{\text{probe}}-1}} \le d_{i_t} \quad \text{for all } t \in \{n_{\text{probe}}, \ldots, n_{\text{list}} - 1\}.$$

The probed set is defined as the first n_{probe} indices,

$$P(q) := (i_0, \ldots, i_{n_{\text{probe}}-1}), \qquad |P(q)| = n_{\text{probe}}.$$

Step 3. ADC lookup tables. For each probed list $i \in P(q)$, the semantics forms the residual query $q - \mu_i$, splits it into M blocks $(q - \mu_i)^{(m)} \in \mathbb{F}^d$, and defines

$$\mathsf{LUT}_{i,m,k} := \text{dist}\big(C_{m,k}, (q - \mu_i)^{(m)}\big), \qquad m \in [M],\ k \in [K].$$

These lookup-table values are arithmetic kernels shared by both circuit instantiations.

Step 4. Candidate distances. For each $i \in P(q)$ and each slot $j \in [n]$, the semantics selects one table entry from each block row and sums them:

$$\widetilde{d}_{i,j} := \sum_{m=0}^{M-1} \mathsf{LUT}_{i,m,\, v^{(m)}_{i,j}}.$$

To ensure that padded slots never affect the output, the semantics fixes a public constant $d_{\max}$ that exceeds any attainable ADC distance of a valid slot under the enforced range bounds, and defines the masked distance

$$D_{i,j} := f_{i,j} \cdot \widetilde{d}_{i,j} + (1 - f_{i,j}) \cdot d_{\max}.$$

All candidate payload-distance pairs are then flattened into

$$S^{\text{orig}}_{\text{item,dist}} := \big((\text{item}_{i,j}, D_{i,j}) \;\big|\; i \in P(q),\ j \in [n]\big),$$

whose length is the fixed scan budget

$$N_{\text{sel}} := n_{\text{probe}} \cdot n.$$

Under the fixed-shape semantics, Step 4 always consumes all n slots of each probed list.

Step 5. Final top-k selection. Finally, the semantics reorders $S^{\text{orig}}_{\text{item,dist}}$ by the distance component and obtains

$$S^{\text{sorted}}_{\text{item,dist}} := \big((\text{item}_0, D_0), (\text{item}_1, D_1), \ldots, (\text{item}_{N_{\text{sel}}-1}, D_{N_{\text{sel}}-1})\big).$$

As in Step 2, only a prefix condition is required:

$$D_0 \le D_1 \le \cdots \le D_{k-1} \quad \text{and}$$
$$D_{k-1} \le D_t \quad \text{for all } t \in \{k, \ldots, N_{\text{sel}} - 1\}.$$

The query output is the first k items in this order:

$$(\mathrm{item}_0, \mathrm{item}_1, \ldots, \mathrm{item}_{k-1}).$$

3 SNAPSHOT BINDING CIRCUIT

This section expands the snapshot-binding part of the proof. The objective is to ensure that every witness value used by the five-step semantics is consistent with the same public commitment

$$\mathrm{com} := (\mathrm{root}_{\mathrm{mk}}, \mathrm{root}_{\mathrm{cb}}).$$

Commitment structure. The hash digest $\mathrm{root}_{\mathrm{cb}}$ binds the PQ codebooks. The Merkle root $\mathrm{root}_{\mathrm{mk}}$ binds the fixed-shape IVF layout. Figure 1 illustrates this Merkle-tree portion of the commitment structure. For each list $i \in [n_{\mathrm{list}}]$ and slot $j \in [n]$, define the slot digest

$$\mathrm{hash}_{i,j} := \mathsf{Hash}\big(i, j, f_{i,j}, \mathrm{item}_{i,j}, v_{i,j}^{\mathrm{PQ}}\big),$$

where $\mathsf{Hash}(\cdot)$ absorbs the M PQ-code components of $v_{i,j}^{\mathrm{PQ}}$ in a fixed canonical order. The Merkle root of list i is

$$\mathrm{root}_i := \mathsf{MerkleTree}(\mathrm{hash}_{i,0}, \ldots, \mathrm{hash}_{i,n-1}).$$

Each centroid is then bound together with its list root via

$$\mathrm{hash}_i := \mathsf{Hash}(i, \mu_i, \mathrm{root}_i),$$

and the top-level IVF-layout root is

$$\mathrm{root}_{\mathrm{mk}} := \mathsf{MerkleTree}(\mathrm{hash}_0, \ldots, \mathrm{hash}_{n_{\mathrm{list}}-1}).$$

What the circuit must bind. The query circuit uses three families of witness values from the snapshot:

- all centroids $\{\mu_i\}_{i \in [n_{\mathrm{list}}]}$ in Steps 1–2;
- all PQ codebooks $\{C_{m,k}\}$ in Step 3;
- all slot records of each probed list in Step 4.

Snapshot binding must guarantee that all of these values come from the same committed snapshot.

Codebook digest check. The circuit recomputes the digest of the witness PQ codebooks from their canonical flattening and enforces equality with the public $\mathrm{root}_{\mathrm{cb}}$. This is the codebook-binding part of the statement.

Centroid and list-digest check. Since centroid distances are computed for all n_{list} lists in Step 1, the circuit cannot authenticate only the probed centroids. Instead, it treats the list roots $\{\mathrm{root}_i\}_{i \in [n_{\mathrm{list}}]}$ as witness digests, recomputes

$$\mathrm{hash}_i = \mathsf{Hash}(i, \mu_i, \mathrm{root}_i), \qquad i \in [n_{\mathrm{list}}],$$

rebuilds the top-level root

$$\mathrm{root}'_{\mathrm{mk}} := \mathsf{MerkleTree}(\mathrm{hash}_0, \ldots, \mathrm{hash}_{n_{\mathrm{list}}-1}),$$

and enforces

$$\mathrm{root}'_{\mathrm{mk}} = \mathrm{root}_{\mathrm{mk}}.$$

This simultaneously binds all centroids and fixes the committed digest of every inverted list.

Opened probed-list check. For each probed list $i \in P(q)$, the circuit recomputes the slot digests

$$\mathrm{hash}_{i,j} = \mathsf{Hash}\big(i, j, f_{i,j}, \mathrm{item}_{i,j}, v_{i,j}^{\mathrm{PQ}}\big), \qquad j \in [n],$$

rebuilds the list root

$$\mathrm{root}'_i := \mathsf{MerkleTree}(\mathrm{hash}_{i,0}, \ldots, \mathrm{hash}_{i,n-1}),$$

forms the corresponding top-level leaf

$$\mathrm{hash}'_i := \mathsf{Hash}(i, \mu_i, \mathrm{root}'_i),$$

and verifies that this leaf opens to $\mathrm{root}_{\mathrm{mk}}$ at position i using a standard Merkle authentication path:

$$\mathsf{MerkleOpen}(\mathrm{hash}'_i, i; \pi_i) = \mathrm{root}_{\mathrm{mk}}.$$

Here π_i denotes the sibling digests along the authentication path for list index i. Concretely, the binary decomposition of i determines at each tree level whether the current digest is placed as the left child or the right child. Starting from hash'_i, the gadget iteratively hashes it with the corresponding sibling digest from π_i according to this left/right pattern, and the final digest is required to equal $\mathrm{root}_{\mathrm{mk}}$.

Why list-level openings suffice. Under the fixed-shape semantics, Step 4 consumes all n slots of every probed list. Therefore, once the circuit has recomputed root'_i from these n opened records, authenticating root'_i to the public top-level root is enough to bind the full list contents used by the query trace. No per-slot authentication paths are required.

Hash-cost abstraction. For gate analysis, we use the standard abstraction

$$G_{\mathsf{Hash}}(L) = \Theta(L),$$

meaning that hashing a sequence of L field elements costs $\Theta(L)$ arithmetic gates. Under this abstraction, building or checking a Merkle tree with m leaves costs linear work in the number of hashed nodes, and a Merkle authentication path of depth $\lceil \log_2 m \rceil$ costs $\Theta(\log m)$ hashes.

Gate complexity. The codebook digest hashes a canonical flattening of all PQ codewords, whose total length is KD, so

$$G_{\mathrm{cb}} = \Theta(KD).$$

For centroid binding, the circuit hashes n_{list} tuples $(i, \mu_i, \mathrm{root}_i)$, each of length $D + 2$, and rebuilds the top-level Merkle tree over these n_{list} leaves. This yields

$$G_{\mathrm{ivf}} = \Theta(n_{\mathrm{list}} D).$$

For one probed list, recomputing the list root requires hashing n slot records. Each slot digest absorbs M PQ-code components

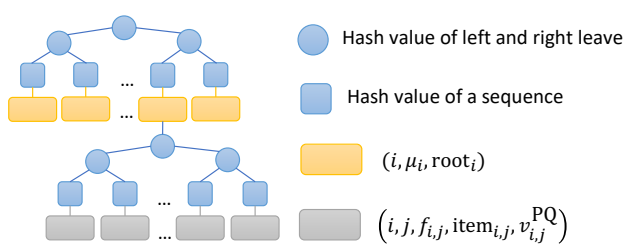

Figure 1: Merkle-tree portion of the snapshot commitment. The figure shows how $\mathrm{root}_{\mathrm{mk}}$ binds the centroid table and the padded inverted lists; the public identifier is $\mathrm{com} = (\mathrm{root}_{\mathrm{mk}}, \mathrm{root}_{\mathrm{cb}})$, where $\mathrm{root}_{\mathrm{cb}}$ separately binds the PQ codebooks.

together with a constant number of extra fields, so the per-list root computation costs

$$G_{\text{list}} = \Theta(nM).$$

Authenticating this list root to the top-level Merkle root additionally requires hashing one top-level leaf $(i, \mu_i, \text{root}'_i)$ and verifying a path of depth $\lceil \log_2 n_{\text{list}} \rceil$, which costs

$$G_{\text{open}} = \Theta(D + \log n_{\text{list}}).$$

Summing the codebook, centroid, and opened-list parts gives

$$\begin{aligned} G_{\text{commit}} &= G_{\text{cb}} + G_{\text{ivf}} + n_{\text{probe}} G_{\text{list}} + n_{\text{probe}} G_{\text{open}} \\ &= \Theta\Big(KD + n_{\text{list}} D + n_{\text{probe}} nM \\ &\qquad + n_{\text{probe}} D + n_{\text{probe}} \log n_{\text{list}}\Big). \end{aligned}$$

This is exactly the snapshot-binding overhead used by the main paper. The later sections add this term to the respective query-semantics costs and then rewrite the totals in the same form as the main paper.

4 GADGETS AND COST MODEL

This section collects the reusable gadgets referenced by the later circuit derivations. We count arithmetic gates over the field $\mathbb{F}$. As in the main paper, all witness values involved in comparisons are range-bounded, and t_{cmp} denotes the bit-length parameter used by the comparison gadget.

Tuple compression Compress. For a tuple $\mathbf{x} = (x_0, \ldots, x_{L-1})$ and a transcript-derived challenge $\beta \in \mathbb{F}$, define

$$\mathsf{Compress}(\mathbf{x}; \beta) := \sum_{t=0}^{L-1} x_t \beta^t.$$

Equivalently, the circuit may evaluate it via Horner's rule:

$$z_{L-1} := x_{L-1}, \qquad z_t := x_t + \beta z_{t+1} \quad \text{for } t = L-2, \ldots, 0,$$

and outputs $\mathsf{Compress}(\mathbf{x}; \beta) := z_0$. This uses $L-1$ multiplications and $L-1$ additions, so

$$G_{\mathsf{Compress}}(L) = 2L - 2 = \Theta(L).$$

Range-bounded comparison Cmp. For bounded field elements $x, y \in [0, 2^{t_{\text{cmp}}-1})$, form

$$\Delta := x - y + 2^{t_{\text{cmp}}-1} \in [0, 2^{t_{\text{cmp}}}).$$

The circuit introduces bits $b_0, \ldots, b_{t_{\text{cmp}}-1}$ and enforces

$$\Delta = \sum_{r=0}^{t_{\text{cmp}}-1} b_r 2^r, \qquad b_r(1 - b_r) = 0 \quad \text{for all } r \in [t_{\text{cmp}}],$$

then defines

$$\mathsf{Cmp}(x, y) := 1 - b_{t_{\text{cmp}}-1}.$$

Under this convention, $\mathsf{Cmp}(x, y) = 1$ means $x < y$, while $\mathsf{Cmp}(x, y) = 0$ means $x \geq y$. This yields

$$G_{\mathsf{Cmp}} = 4t_{\text{cmp}} + 2 = \Theta(t_{\text{cmp}}).$$

Bubble-pass gadget BubblePass. For a sequence $S = (s_0, \ldots, s_{L-1})$, one bubble pass scans the sequence from right to left. Let $\widehat{s}_{L-1} := s_{L-1}$. For each $r = L-2, \ldots, 0$, the circuit enforces

$$\begin{cases} b_r := \mathsf{Cmp}(\widehat{s}_{r+1}, s_r), \\ s''_{r+1} := (1 - b_r)\widehat{s}_{r+1} + b_r s_r, \\ \widehat{s}_r := b_r \widehat{s}_{r+1} + (1 - b_r) s_r. \end{cases}$$

Finally, it sets $s''_0 := \widehat{s}_0$ and outputs both the swap bits $(b_0, \ldots, b_{L-2})$ and the bubbled sequence $(s''_0, \ldots, s''_{L-1})$. Each position contributes one comparison and two conditional assignments, so

$$\begin{aligned} G_{\mathsf{BubblePass}}(L) &= (L-1)\big(G_{\mathsf{Cmp}} + 8\big) \\ &= 4Lt_{\text{cmp}} + 10L - 4t_{\text{cmp}} - 10 \\ &= \Theta(L\, t_{\text{cmp}}). \end{aligned}$$

Parallel permutation Permute. Given the swap bits $(b_0, \ldots, b_{L-2})$ returned by BubblePass and a parallel sequence $P = (p_0, \ldots, p_{L-1})$, the circuit applies the same swaps to P. Let $\widehat{p}_{L-1} := p_{L-1}$. For each $r = L-2, \ldots, 0$, it enforces

$$\begin{cases} p''_{r+1} := (1 - b_r)\widehat{p}_{r+1} + b_r p_r, \\ \widehat{p}_r := b_r \widehat{p}_{r+1} + (1 - b_r) p_r. \end{cases}$$

and finally sets $p''_0 := \widehat{p}_0$. Since each position uses only two conditional assignments,

$$G_{\mathsf{Permute}}(L) = 8(L-1) = 8L - 8 = \Theta(L).$$

Multiset equality SetEq. After tuple compression, multiset equality between two size-L sequences

$$X = \{x_0, \ldots, x_{L-1}\}, \qquad Y = \{y_0, \ldots, y_{L-1}\}$$

is enforced by the polynomial identity

$$\prod_{t=0}^{L-1} (\alpha - x_t) = \prod_{t=0}^{L-1} (\alpha - y_t),$$

where $\alpha \in \mathbb{F}$ is a random challenge. Evaluating each side requires L subtractions and $L-1$ multiplications, so

$$G_{\mathsf{SetEq}}(L) = 4L - 2 = \Theta(L).$$

Multiset inclusion Incl. To prove multiset inclusion $X \subset_M Y$ with multiplicities, let

$$n_{\max} := \max\{|X|, |Y|\}.$$

Pad both multisets to length $n_{\max}$, obtaining X' and Y'. For simplicity, the padding may repeat the 0-th value of each multiset:

$$X' = X \cup^{n_{\max} - |X|} \{x_0\}, \qquad Y' = Y \cup^{n_{\max} - |Y|} \{y_0\}.$$

The prover then supplies witness sequences

$$X'' = (x''_0, \ldots, x''_{n_{\max}-1}), \qquad Y'' = (y''_0, \ldots, y''_{n_{\max}-1}),$$

where X'' is the nondecreasing ordering of X' and Y'' is aligned with X'' while preserving the multiset of Y'. The alignment rule is the following:

$$x''_0 = y''_0,$$

and whenever the value of X'' changes at position r, the corresponding value of Y'' must match it as well. The remaining positions of Y'' may be filled with the leftover elements of Y' in arbitrary order.

For example, let

$$X = [1, 0, 2, 2], \qquad Y = [0, 1, 2, 3, 4, 5].$$

After padding by repeating the 0-th value, one valid choice is

$$X' = [1, 0, 2, 2, 1, 1], \qquad Y' = [0, 1, 2, 3, 4, 5].$$

Gadget	Gate complexity	Gadget	Gate complexity
Compress	$\Theta(L)$	SetEq	$\Theta(L)$
Cmp	$\Theta(t_{\text{cmp}})$	Incl	$\Theta(t_{\text{cmp}}\, n_{\max})$
BubblePass	$\Theta(L\, t_{\text{cmp}})$	LookUp	$\Theta(L)$
Permute	$\Theta(L)$	—	—

Table 1: Reusable gadgets and their asymptotic arithmetic-gate complexity. Here L denotes the tuple, list, or table length, and $n_{\max} = \max\{|X|, |Y|\}$ for Incl.

Sorting X' gives

$$X'' = [0, 1, 1, 1, 2, 2].$$

Then one valid aligned witness for Y' is

$$Y'' = [0, 1, 3, 4, 2, 5],$$

because $Y''_0 = X''_0$, and at the change points of X'' we again have matching values, namely $Y''_1 = X''_1 = 1$ and $Y''_4 = X''_4 = 2$.

The circuit enforces

$$\begin{cases} \mathsf{SetEq}(X', X''; \alpha_0), \\ \mathsf{SetEq}(Y', Y''; \alpha_1), \\ \mathsf{Cmp}(x''_{r+1}, x''_r) = 0, \quad r \in [n_{\max} - 1], \\ x''_0 = y''_0, \\ (x''_r - x''_{r-1})(x''_r - y''_r) = 0, \quad r \in \{1, \dots, n_{\max} - 1\}. \end{cases}$$

The first two lines bind the witnesses to the padded multisets, the third line enforces the sorted order of X'', and the last two lines encode the alignment rule. In particular, for each $r \geq 1$, the constraint

$$(x''_r - x''_{r-1})(x''_r - y''_r) = 0$$

means that either $x''_r = x''_{r-1}$, so the current position stays inside the same value block of X'', or $x''_r = y''_r$, so the first position of a new value block in X'' must be matched by the same value in Y''. The resulting gate count is

$$\begin{aligned} G_{\mathsf{Incl}}(n_{\max}) &= 2G_{\mathsf{SetEq}}(n_{\max}) + (n_{\max} - 1)G_{\mathsf{Cmp}} + 3(n_{\max} - 1) \\ &= 4t_{\text{cmp}} n_{\max} + 13 n_{\max} - 4t_{\text{cmp}} - 9 \\ &= \Theta(t_{\text{cmp}}\, n_{\max}). \end{aligned}$$

In-circuit lookup LookUp. For a length-L table $(s_0, \dots, s_{L-1})$ and an index k^*, the gadget returns

$$s = \mathsf{LookUp}\big(k^*, (s_0, \dots, s_{L-1})\big).$$

The circuit introduces one-hot selector variables $\lambda_0, \dots, \lambda_{L-1}$ and enforces

$$\begin{cases} \lambda_\ell(1 - \lambda_\ell) = 0, \quad \ell \in [L], \\ \sum_{\ell=0}^{L-1} \lambda_\ell = 1, \\ \sum_{\ell=0}^{L-1} \lambda_\ell\, \ell = k^*, \\ s = \sum_{\ell=0}^{L-1} \lambda_\ell s_\ell. \end{cases}$$

This yields

$$G_{\mathsf{LookUp}}(L) = 7L - 3 = \Theta(L).$$

5 CIRCUIT-ONLY DESIGN

This section gives a step-by-step expansion of the circuit-only baseline. The baseline keeps all data-dependent operations inside the circuit. Accordingly, the dominant costs come from explicit comparison networks and in-circuit lookup selection. We refer to Section 4 for the reusable gadgets used below, in particular BubblePass, Permute, and LookUp.

Step 1. Centroid distances. The circuit recomputes

$$d_i := \mathsf{dist}(q, \mu_i), \qquad i \in [n_{\text{list}}],$$

and materializes

$$S^{\text{orig}}_{\text{idx,dist}} = \big((0, d_0), (1, d_1), \dots, (n_{\text{list}} - 1, d_{n_{\text{list}}-1})\big).$$

Each centroid distance is a standard fixed-point arithmetic kernel over D coordinates, so the total cost of Step 1 is

$$G_{\text{step1}} = \Theta(n_{\text{list}} D).$$

Step 2. Probe selection by comparison passes. The circuit maintains two aligned length-n_{list} arrays: the distance array $(d_0, \dots, d_{n_{\text{list}}-1})$ and the parallel index array $(0, 1, \dots, n_{\text{list}} - 1)$. One BubblePass pass applies bounded comparisons to adjacent distance entries and produces swap bits; the same swap bits are then applied to the parallel index array through Permute.

The baseline executes n_{probe} such passes. After these passes, the first n_{probe} positions satisfy the Step 2 prefix-order and boundary conditions, and the first n_{probe} indices define $P(q)$. The cost is therefore

$$\begin{aligned} G_{\text{step2}} &= n_{\text{probe}}\Big(G_{\mathsf{BubblePass}}(n_{\text{list}}) + G_{\mathsf{Permute}}(n_{\text{list}})\Big) \\ &= \Theta\big(n_{\text{probe}} n_{\text{list}} t_{\text{cmp}}\big). \end{aligned}$$

Step 3. ADC lookup tables. Once $P(q)$ is fixed, the circuit constructs

$$\mathsf{LUT}_{i,m,k} = \mathsf{dist}\big(C_{m,k}, (q - \mu_i)^{(m)}\big), \qquad i \in P(q),\ m \in [M],\ k \in [K].$$

For one fixed cluster i, the table has MK entries, and each entry is a distance computation over one block of dimension $d = D/M$. Hence the cost per probed cluster is

$$\Theta(MK \cdot d) = \Theta(KD),$$

and over all probed clusters we obtain

$$G_{\text{step3}} = \Theta(n_{\text{probe}} KD).$$

Step 4. Candidate scoring by in-circuit lookup. For each $i \in P(q)$, each slot $j \in [n]$, and each sub-quantizer $m \in [M]$, the circuit must select

$$\ell^{(m)}_{i,j} := \mathsf{LUT}_{i,m,v^{(m)}_{i,j}}.$$

The baseline realizes this selection through an in-circuit LookUp over the length-K row

$$(\mathsf{LUT}_{i,m,0}, \dots, \mathsf{LUT}_{i,m,K-1}).$$

After all selected entries are obtained, the circuit computes

$$\widetilde{d}_{i,j} = \sum_{m=0}^{M-1} \ell^{(m)}_{i,j}$$

and then applies the validity mask

$$D_{i,j} = f_{i,j} \cdot \widetilde{d}_{i,j} + (1 - f_{i,j}) \cdot d_{\max}.$$

The arithmetic sum-and-mask part is linear in M and does not dominate. The dominant work comes from the $n_{\text{probe}} nM$ lookup calls, each of cost $\Theta(K)$, so

$$G_{\text{step4}} = \Theta(n_{\text{probe}} nMK).$$

Step 5. Final top-k extraction by comparison passes. The circuit flattens the candidate list into a length-N_{sel} distance array and a parallel payload array, where

$$N_{\text{sel}} = n_{\text{probe}} n.$$

It then applies k comparison-network passes to the candidate distances and synchronously permutes the payloads with the same swap bits. After k passes, the first k positions satisfy the Step 5 prefix-order and boundary conditions, and the first k payloads are returned.

The cost is

$$\begin{aligned} G_{\text{step5}} &= k\Big(G_{\text{BubblePass}}(N_{\text{sel}}) + G_{\text{Permute}}(N_{\text{sel}})\Big) \\ &= \Theta(k n_{\text{probe}} n t_{\text{cmp}}). \end{aligned}$$

Baseline query-circuit complexity. Collecting the five steps yields

$$\begin{aligned} G_{\text{baseline}} = \Theta\Big(& n_{\text{list}} D + n_{\text{probe}} n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} KD \\ & + n_{\text{probe}} nMK + k n_{\text{probe}} n t_{\text{cmp}} \Big). \end{aligned}$$

The dominant contributors are exactly the data-dependent primitives left inside the circuit: comparison networks in Steps 2 and 5, and random-access selection in Step 4. Adding the snapshot-binding overhead from the previous section gives

$$\begin{aligned} G^{\text{total}}_{\text{baseline}} &= G_{\text{baseline}} + G_{\text{commit}} \\ &= \Theta\Big(KD + n_{\text{list}} D + n_{\text{probe}} nM \\ &\quad + n_{\text{probe}} D + n_{\text{probe}} \log n_{\text{list}} + n_{\text{list}} D \\ &\quad + n_{\text{probe}} n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} KD \\ &\quad + n_{\text{probe}} nMK + k n_{\text{probe}} n t_{\text{cmp}} \Big). \end{aligned}$$

To match the presentation in the main paper, we keep the additive Merkle-opening term explicit and rewrite the total as

$$\begin{aligned} G^{\text{total}}_{\text{baseline}} = \Theta\Big(& n_{\text{list}} D + n_{\text{probe}} n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} KD \\ & + n_{\text{probe}} nMK + k n_{\text{probe}} n t_{\text{cmp}} \\ & + n_{\text{probe}} \log n_{\text{list}} \Big). \end{aligned}$$

Asymptotically, the last term is subsumed by the comparison-network term $n_{\text{probe}} n_{\text{list}} t_{\text{cmp}}$, but keeping it explicit makes the source of the snapshot-binding overhead transparent.

6 MULTISET-BASED DESIGN

This section expands the multiset-based instantiation. The arithmetic kernels remain the same as in the baseline. What changes is only how the circuit justifies ordering and lookup relations: instead of realizing sorting and random access directly in-circuit, the prover supplies the corresponding witness objects and the circuit checks them through multiset relations plus lightweight boundary conditions. We refer to Section 4 for the reusable gadgets used below, in particular Compress, SetEq, Incl, and Cmp.

Shared arithmetic kernels. As in the baseline, the circuit still recomputes all centroid distances d_i in Step 1 and all ADC lookup tables $\mathsf{LUT}_{i,m,k}$ in Step 3. The arithmetic sum-and-mask part of Step 4 is also unchanged. Therefore, the arithmetic kernel costs remain

$$\Theta(n_{\text{list}} D) \qquad \text{and} \qquad \Theta(n_{\text{probe}} KD).$$

Step 2. Probe selection via multiset equality and boundary checks. The prover supplies the witness sequence

$$S^{\text{sorted}}_{\text{idx,dist}} = \big((i_0, d_{i_0}), (i_1, d_{i_1}), \ldots, (i_{n_{\text{list}}-1}, d_{i_{n_{\text{list}}-1}})\big).$$

The circuit first compresses each pair (i_t, d_{i_t}) and checks

$$\mathsf{SetEq}\big(S^{\text{sorted}}_{\text{idx,dist}}, S^{\text{orig}}_{\text{idx,dist}}\big).$$

This guarantees that the witness sequence is a valid reordering of the circuit-computed base sequence and that each claimed distance is tied to the corresponding d_i.

The circuit then enforces the Step 2 prefix-order and boundary conditions using Cmp:

$$d_{i_0} \le \cdots \le d_{i_{n_{\text{probe}}-1}}, \qquad d_{i_{n_{\text{probe}}-1}} \le d_{i_t}$$

for all $t \in \{n_{\text{probe}}, \ldots, n_{\text{list}} - 1\}$. The multiset-equality term is linear in n_{list}, while the comparisons cost

$$(n_{\text{list}} - 1) \cdot \Theta(t_{\text{cmp}}).$$

Hence

$$G^{\text{ms}}_{\text{step2}} = \Theta(n_{\text{list}} t_{\text{cmp}}).$$

Step 4. Candidate scoring via multiset inclusion. Instead of performing one in-circuit lookup for every PQ index, the prover supplies the selected entries

$$\ell^{(m)}_{i,j} := \mathsf{LUT}_{i,m,v^{(m)}_{i,j}}.$$

The circuit proves that these witness values are consistent with the recomputed LUT tables by enforcing a multiset inclusion relation between the selected-entry tuples

$$X = \big\{(i, m, v^{(m)}_{i,j}, \ell^{(m)}_{i,j}) \;\big|\; i \in P(q),\ j \in [n],\ m \in [M]\big\}$$

and the full LUT tuples

$$Y = \big\{(i, m, k, \mathsf{LUT}_{i,m,k}) \;\big|\; i \in P(q),\ m \in [M],\ k \in [K]\big\}.$$

The two multisets have sizes

$$|X| = n_{\text{probe}} nM, \qquad |Y| = n_{\text{probe}} MK,$$

so the inclusion gadget operates on

$$n_{\max} = n_{\text{probe}} M \max\{n, K\}$$

elements. Its cost is therefore

$$G_{\text{incl-part}} = \Theta\big(t_{\text{cmp}}\, n_{\text{probe}} M \max\{n, K\}\big).$$

After this inclusion check, the circuit computes

$$\widetilde{d}_{i,j} = \sum_{m=0}^{M-1} \ell^{(m)}_{i,j}$$

and

$$D_{i,j} = f_{i,j} \cdot \widetilde{d}_{i,j} + (1 - f_{i,j}) \cdot d_{\max}$$

exactly as in the baseline. These linear arithmetic operations do not dominate the inclusion term, so

$$G^{\text{ms}}_{\text{step4}} = \Theta\big(t_{\text{cmp}}\, n_{\text{probe}} M \max\{n, K\}\big).$$

Step 5. Final top-k selection via multiset equality and boundary checks. The prover supplies the witness sequence

$$S^{\text{sorted}}_{\text{item,dist}} = \big((\text{item}_0, D_0), (\text{item}_1, D_1), \ldots, (\text{item}_{N_{\text{sel}}-1}, D_{N_{\text{sel}}-1})\big).$$

The circuit compresses each payload-distance pair, checks

$$\mathsf{SetEq}\big(S^{\text{sorted}}_{\text{item,dist}}, S^{\text{orig}}_{\text{item,dist}}\big),$$

and then enforces the Step 5 prefix-order and boundary conditions:

$$D_0 \le \cdots \le D_{k-1}, \qquad D_{k-1} \le D_t$$

for all $t \in \{k, \ldots, N_{\text{sel}} - 1\}$. The multiset-equality term is linear in N_{sel}, while the comparisons cost

$$(N_{\text{sel}} - 1) \cdot \Theta(t_{\text{cmp}}).$$

Since $N_{\text{sel}} = n_{\text{probe}} n$, this yields

$$G^{\text{ms}}_{\text{step5}} = \Theta(n_{\text{probe}} n t_{\text{cmp}}).$$

Multiset query-circuit complexity. Collecting the shared arithmetic kernels and the multiset-based enforcement terms gives

$$G_{\text{multiset}} = \Theta\Big(n_{\text{list}} D + n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} K D + t_{\text{cmp}}\, n_{\text{probe}} M \max\{n, K\} + n_{\text{probe}} n t_{\text{cmp}}\Big).$$

Adding snapshot binding gives

$$\begin{aligned} G^{\text{total}}_{\text{multiset}} &= G_{\text{multiset}} + G_{\text{commit}} \\ &= \Theta\Big(KD + n_{\text{list}} D + n_{\text{probe}} n M \\ &\quad + n_{\text{probe}} D + n_{\text{probe}} \log n_{\text{list}} + n_{\text{list}} D \\ &\quad + n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} K D \\ &\quad + t_{\text{cmp}}\, n_{\text{probe}} M \max\{n, K\} + n_{\text{probe}} n t_{\text{cmp}}\Big). \end{aligned}$$

To match the presentation in the main paper, we keep the additive Merkle-opening term explicit and rewrite the total as

$$\begin{aligned} G^{\text{total}}_{\text{multiset}} = \Theta\Big(&n_{\text{list}} D + n_{\text{list}} t_{\text{cmp}} + n_{\text{probe}} K D \\ &+ t_{\text{cmp}}\, n_{\text{probe}} M \max\{n, K\} \\ &+ n_{\text{probe}} n t_{\text{cmp}} + n_{\text{probe}} \log n_{\text{list}}\Big). \end{aligned}$$